Prepare Level 2: The Night Watchman

```
Document:    Prepare Level 2: The Night Watchman
Version:     v1.0.0
Author:      Celaya Solutions
Contact:     hello@celayasolutions.com
Date:        2026-09-06
SHA256:      ed86a594589e6c97b62deec895fc00c96a607bafe78f95e6fa0df8c659093364
Chain:       n/a
Tx:          [not anchored]
License:     All Rights Reserved / Celaya Solutions
```

Finish this setup before the 90-minute meeting. Allow 20-30 minutes; record your actual time. Use your own Windows or Mac computer and the same personal course fork used in Level 1. A phone, partner, or saved packet supports participation but does not prove you launched your own project.

Use the instructor's reviewed Level 2 course copy. The old Level 1 release, v1.0.0-rc.1, does not contain the new watcher commands. If `projects/watchman/` is missing, stop and ask for the Level 2 copy. Do not combine files from different releases.

1. Open the right accounts and folder

Where: Browser and GitHub Desktop. **Action:** Sign in to GitHub. Open your existing course fork, then open that same repository in GitHub Desktop. Select **Repository > View on GitHub** and check that the owner is you. In GitHub's **Code** tab, note the default branch shown when you open the repository home. For this lesson, use a public personal fork with Issues enabled.

Why: Your watcher must act in your practice space. **Expected result:** The browser and Desktop show the same owner/repository. **Recovery:** If the owner is `celaya-solutions`, return to your own fork. New learners use the shared Windows guide or Mac guide for accounts, GitHub Desktop, uv, and cloning; the instructor supplies the Level 1 catch-up activity. No new model download or API billing is needed for Level 2.

Where: Desktop's **Current branch** menu. **Action:** Select your fork's default branch; choose **Fetch origin**, then **Pull origin** if shown. If you have unfinished edits, keep them and ask the instructor to help switch safely. Check that `projects/watchman/watch.py`, `projects/watchman/workflow.template.yml`, and the Level 2 lesson are present in the reviewed copy.

Why: Hosted manual controls and the daily schedule need the installed workflow on the default branch. **Expected result:** Your copy includes the complete Level 2 project. **Recovery:** If upstream has a newer instructor-approved release, use your fork's **Sync fork > Update branch**, then fetch/pull in Desktop. Do not discard your Level 1 changes to solve a conflict; use instructor help. The maintainer must publish this candidate before this update route can supply it.

2. Open a terminal in the course folder

Windows: In Desktop choose **Repository > Show in Explorer**. Click Explorer's address bar, type `powershell`, and press Enter. In the new window run `Get-Location`, then `Get-Item pyproject.toml`.

Mac: In Desktop choose **Repository > Show in Finder**. Press Command+Up to show the folder that contains the selected course folder. Open Terminal from Applications > Utilities. Type `cd` with a space, drag the course folder from Finder into Terminal, and press Return. Run `pwd`, then `ls pyproject.toml`.

Why: Commands must run from the repository root. **Expected result:** The path is your course clone and `pyproject.toml` exists. **Recovery:** If it is missing, repeat the folder steps. Never type a sample username or guessed path.

Run each command separately and wait for it to finish:

```
uv sync --frozen
uv run --frozen zta doctor watchman
uv run --frozen zta test watchman
```

Why: Install the locked tools, confirm project files, and test behavior before allowing writes. **Expected result:** The doctor prints `PASS: local watcher files found`; all watcher tests pass. **Recovery:** For `uv` not found, reopen the terminal after the shared OS setup. For a missing project or failing test, keep the error and use the recovery card. Never enter an API key to fix a watcher check.

3. Install the workflow and private worksheet

Where: The same terminal. **Action:** Run:

```
uv run --frozen zta prepare watchman
```

Why: This copies the reviewed template to GitHub's exact workflow folder and makes a private worksheet. **Expected result:** `.github/workflows/watchman.yml` and `.zta/PROJECT-LAB-02.md` are created. Existing files are kept. This command neither publishes nor enables anything. **Recovery:** If it says an existing workflow was kept, compare it with `projects/watchman/workflow.template.yml` using the instructor; do not overwrite unexplained edits.

Where: GitHub Desktop's **Changes** tab. **Action:** Review `.github/workflows/watchman.yml`. It requests only `contents: write` and `issues: write` for the watcher job. Enter the summary `Install the Level 2 practice watcher`, commit to your fork's default branch, then click **Push origin**.

Why: GitHub cannot run a file that exists only on your laptop. **Expected result:** Your fork's Code tab shows `.github/workflows/watchman.yml` on its default branch. No `.zta/`, private proof, or key appears in Changes. **Recovery:** If Desktop shows a permission or branch-rule block, stop and ask the instructor; do not broaden organization permissions or paste a personal token. If on a feature branch, review and merge the workflow into your own default branch with instructor help before continuing.

4. Check GitHub controls while the watcher is off

Where: Your fork's **Settings > General > Features**. **Action:** Enable **Issues** if unchecked. Then open **Actions** and enable workflows in your own fork if GitHub presents that choice. Select **Project Lab Watchman** in the left sidebar.

Why: Forks may start with Actions or Issues disabled. **Expected result:** You can see the workflow and **Run workflow** control. **Recovery:** Missing control: check the exact file path, default branch, repository ownership, and the `workflow_dispatch` section. Ask the instructor about organization policy; do not enable write tokens for pull requests. See GitHub's manual-run guide.

Where: Settings > Secrets and variables > Actions > Variables. **Action:** Confirm there is no repository variable named `WATCHMAN_ENABLED` with value `true`. If it exists from a past exercise, edit its value to `false`. This is a non-secret on/off value, not a credential.

Why: The template is present but its job stays off until class. **Expected result:** No practice fetch or issue is made by a scheduled job while this value is absent or false. **Recovery:** If a run is already queued or active, cancel it from Actions too. Switching off future triggers does not undo work already running.

Ready check

- I can identify my own fork and its default branch.

- The local watcher tests pass; I recorded their count and my setup time.

- The workflow is committed to that branch; Issues and Actions controls are visible.

- The practice page contains `OPEN`; there is no existing `.watch-state/watchman.json` in this fresh practice setup. If there is, preserve it and ask the instructor before class.

- `WATCHMAN_ENABLED` is absent or false. No hosted run has been started for the exercise.

- I can open `.zta/PROJECT-LAB-02.md` in my text editor. It is ignored by Git.

- I can sign in to the course and find Level 2, or I have recorded the access block.

After two failed attempts or five minutes on one block, record it and use the saved packet. Mark the route honestly and arrange the missing live check later. Bring the lesson, worksheet, and manual card offline if needed.